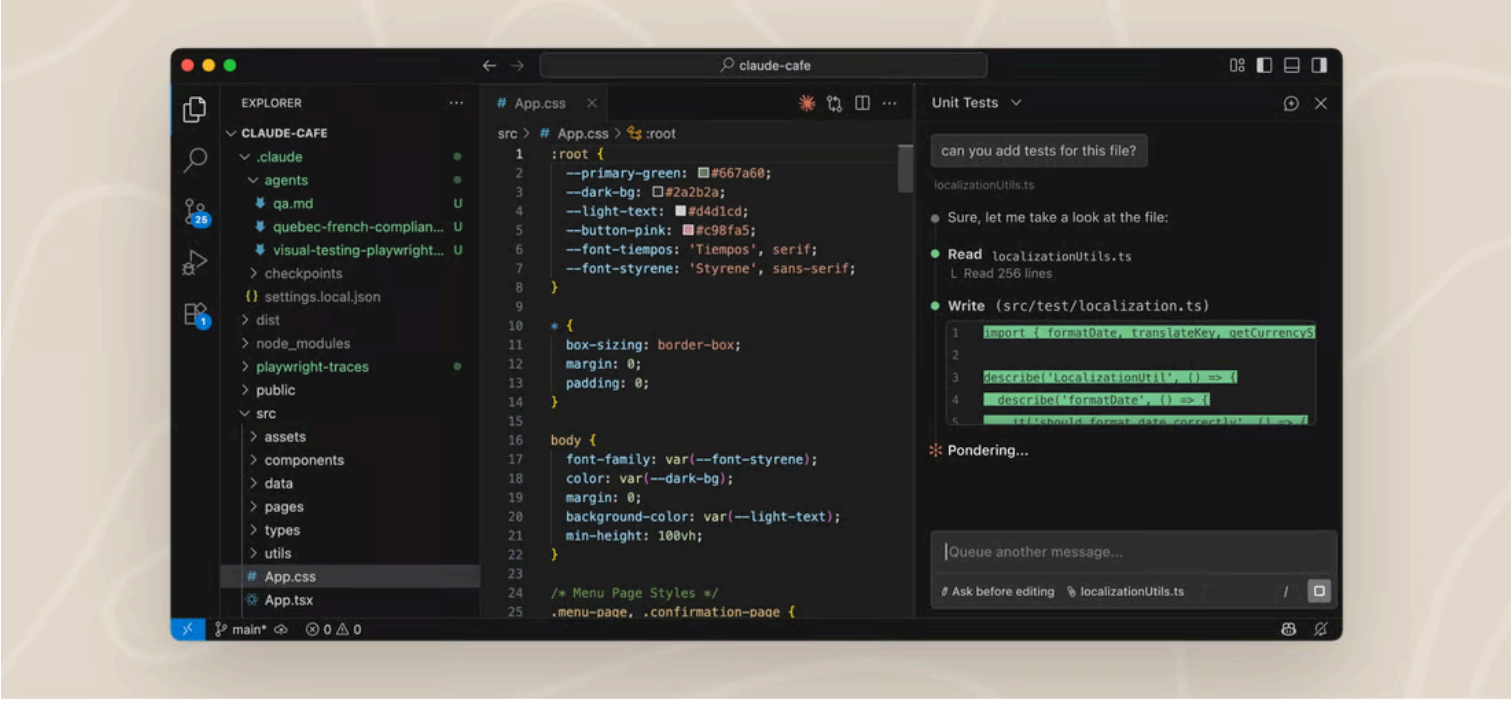
PLATFORMS AND INTEGRATIONS

Use Claude Code in VS Code

Install and configure the Claude Code extension for VS Code. Get AI coding assistance with inline diffs, @-mentions, plan review, and keyboard shortcuts.



The VS Code extension provides a native graphical interface for Claude Code, integrated directly into your IDE. This is the recommended way to use Claude Code in VS Code.

With the extension, you can review and edit Claude's plans before accepting them, auto-accept edits as they're made, @-mention files with specific line ranges from your selection, access conversation history, and open multiple conversations in separate tabs or windows.

Prerequisites

Before installing, make sure you have:

- VS Code 1.98.0 or higher
- An Anthropic account: any paid Claude subscription (Pro, Max, Team, or Enterprise) or a Claude Console account works, and no API key is required. You'll **sign in** with this account when you first open the extension. If you access Claude through a third-party provider like Amazon Bedrock or

Google Vertex AI, see **Use third-party providers** for setup instructions.

The extension bundles its own copy of the CLI (command-line interface) for the chat panel. To run `claude` in VS Code's integrated terminal, you also need the **standalone CLI install**. See **VS Code extension vs. Claude Code CLI** for details.

Install the extension

Click the link for your IDE to install directly:

- **Install for VS Code**
- **Install for Cursor**

Or in VS Code, press `Cmd+Shift+X` (Mac) or `Ctrl+Shift+X` (Windows/Linux) to open the Extensions view, search for "Claude Code", and click **Install**.

The extension also installs in other VS Code forks like Devin Desktop or Kiro. Search for "Claude Code" in the editor's Extensions view, or install from the **Open VSX registry**. If your editor can't install the extension, **install the CLI** and run `claude` in its integrated terminal instead. The CLI works in any terminal.

If the extension doesn't appear after installation, restart VS Code or run "Developer: Reload Window" from the Command Palette.

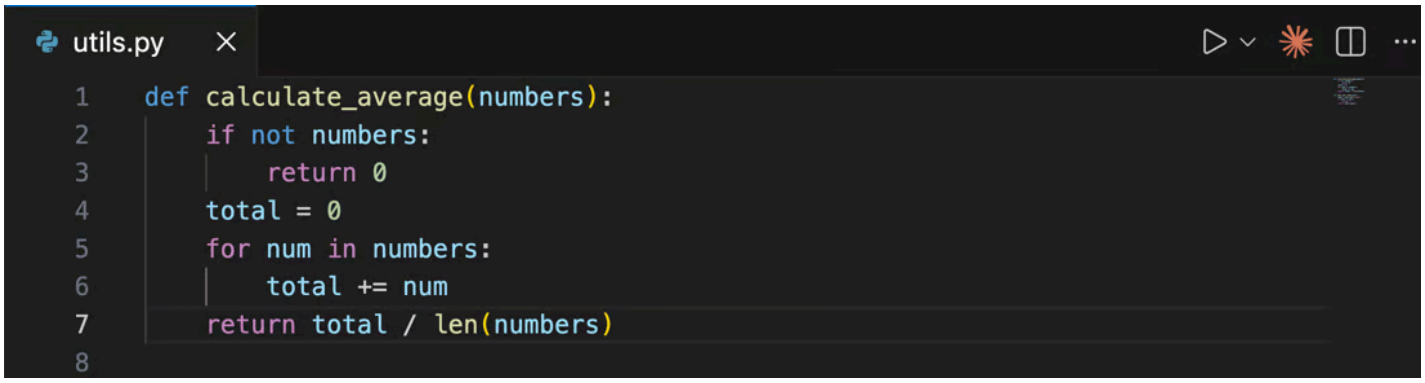
Get started

Once installed, you can start using Claude Code through the VS Code interface:

1 Open the Claude Code panel

Throughout VS Code, the Spark icon indicates Claude Code: ✳

The quickest way to open Claude is to click the Spark icon in the **Editor Toolbar** (top-right corner of the editor). The icon only appears when you have a file open.



Other ways to open Claude Code:

- **Activity Bar**: click the Spark icon in the left sidebar to open the sessions list. Click any session to open it as a full editor tab, or start a new one. This icon is always visible in the Activity Bar.
- **Command Palette**: `Cmd+Shift+P` (Mac) or `Ctrl+Shift+P` (Windows/Linux), type "Claude Code", and select an option like "Open in New Tab"
- **Status Bar**: click ✱ **Claude Code** in the bottom-right corner of the window. This works even when no file is open.

You can drag the Claude panel to reposition it anywhere in VS Code. See **Customize your workflow** for details.

2 Sign in

The first time you open the panel, a sign-in screen appears. Click **Sign in** and complete authorization in your browser.

If you see **Not logged in · Please run /login** later, the extension reopens the sign-in screen automatically. If it doesn't appear, reload the window from the Command Palette with **Developer: Reload Window**.

If you have `ANTHROPIC_API_KEY` set in your shell but still see the sign-in prompt, VS Code may not have inherited your shell environment. Launch VS Code from a terminal with `code .` so it inherits your environment variables, or sign in with your Claude account instead.

After you sign in, a **Learn Claude Code** checklist appears. Work through each item by clicking **Show me**, or dismiss it with the X. To reopen it later, uncheck **Hide Onboarding** in VS Code settings under Extensions → Claude Code.

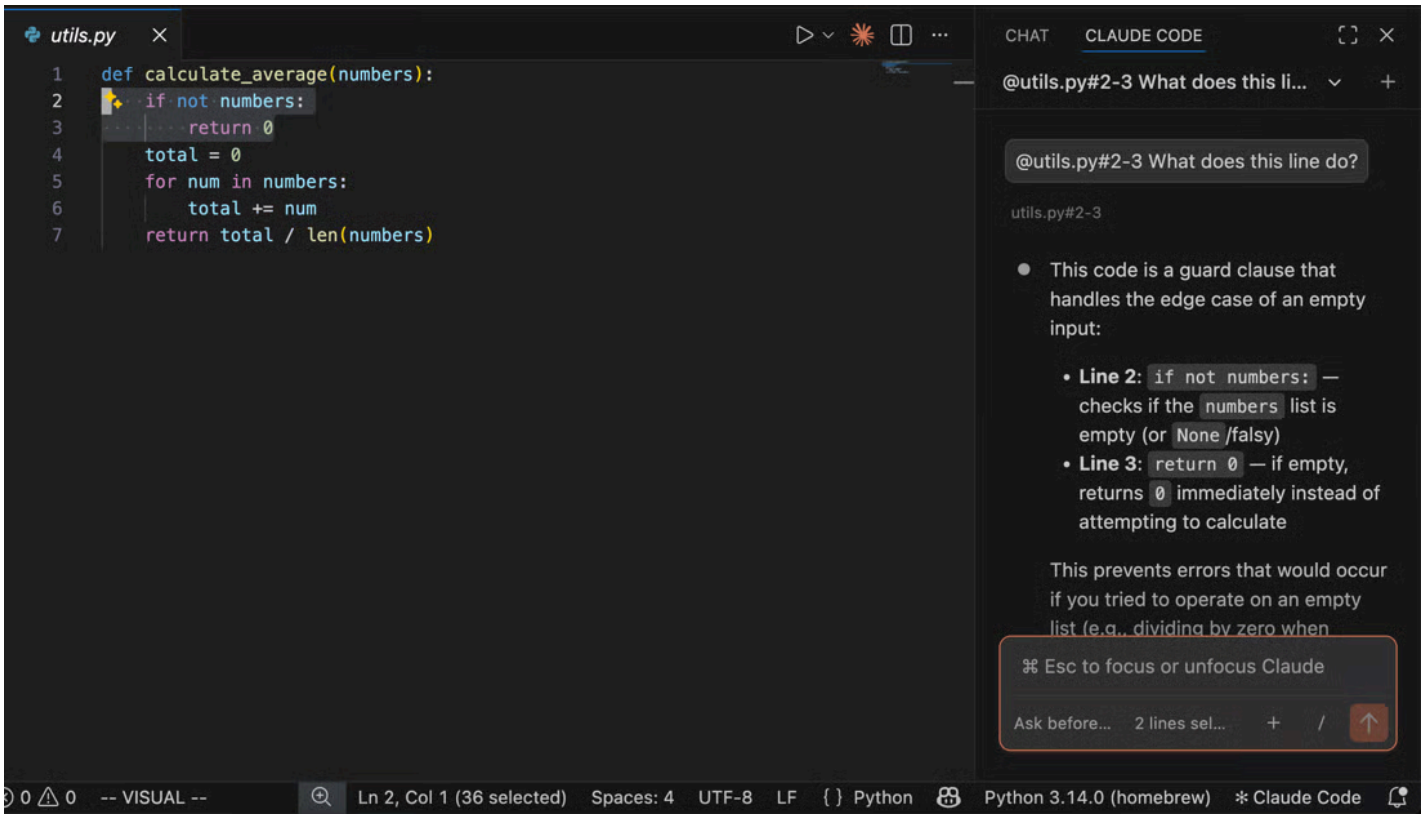
3 Send a prompt

Ask Claude to help with your code or files, whether that's explaining how something works,

debugging an issue, or making changes.

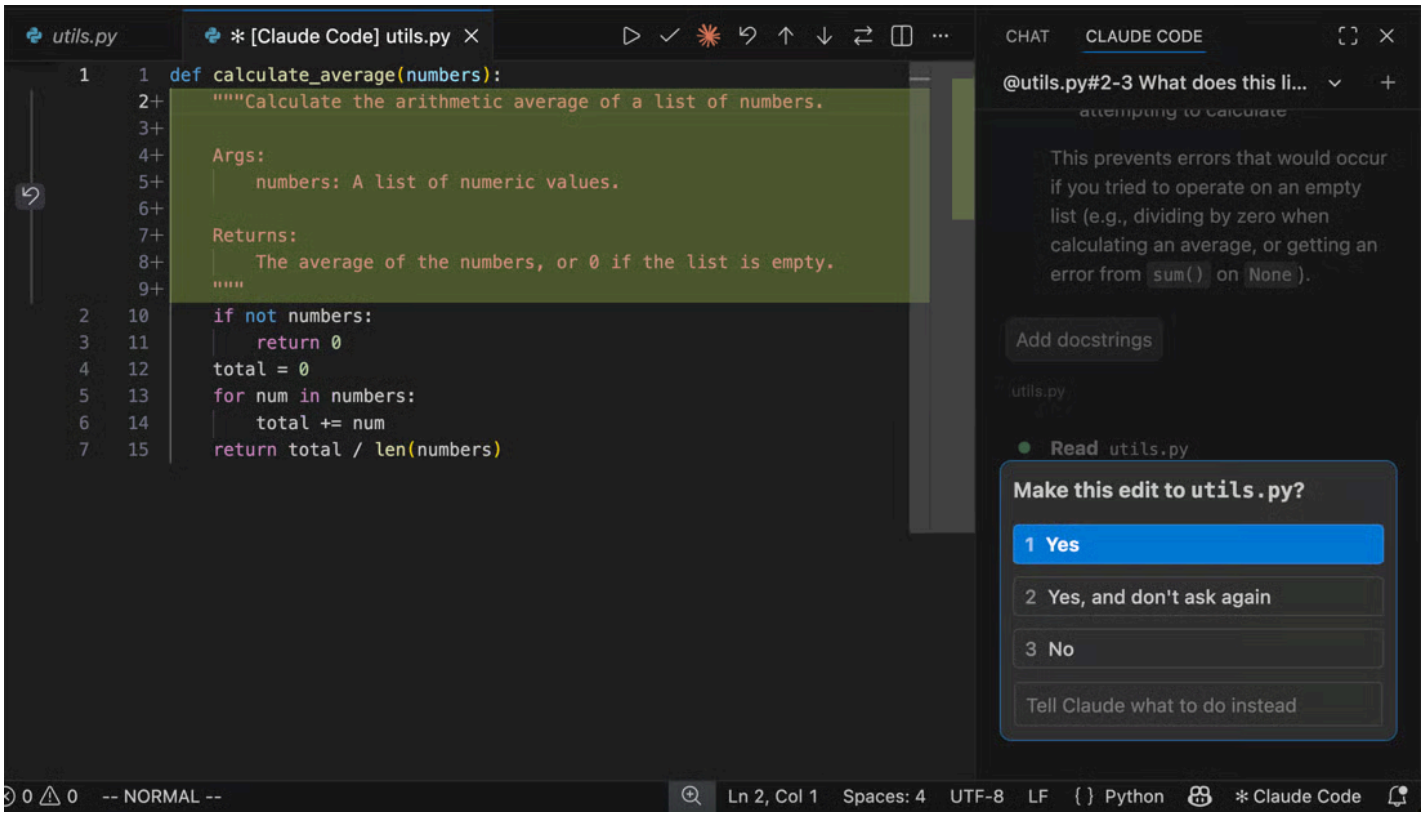
> Claude automatically sees your selected text. Press `Option+K` (Mac) / `Alt+K` (Windows/Linux) to also insert an @-mention reference (like `@file.ts#5-10`) into your prompt.

Here's an example of asking about a particular line in a file:



4 Review changes

When Claude wants to edit a file, it shows a side-by-side comparison of the original and proposed changes, then asks for permission. You can accept, reject, or tell Claude what to do instead. If you edit the proposed content directly in the diff view before accepting, Claude is told that you modified it so it does not assume the file matches its original proposal.



For more ideas on what you can do with Claude Code, see **Common workflows**.

Run “Claude Code: Open Walkthrough” from the Command Palette for a guided tour of the basics.

Use the prompt box

The prompt box supports several features:

- **Permission modes**: click the mode indicator at the bottom of the prompt box to switch modes. In normal mode, Claude asks permission before each action. In Plan mode, Claude describes what it will do and waits for approval before making changes. VS Code automatically opens the plan as a full markdown document where you can add inline comments to give feedback before Claude begins. In auto-accept mode, Claude makes edits without asking. Set the default in VS Code settings under `claudeCode.initialPermissionMode`.
- **Command menu**: click `/` or type `/` to open the command menu. Options include attaching files, switching models, toggling extended thinking, viewing plan usage (`/usage`), and starting a **Remote Control** session (`/remote-control`). The Customize section provides access to MCP servers, hooks, memory, permissions, and plugins. Items with a terminal icon open in the integrated terminal.
- **Context indicator**: the prompt box shows how much of Claude’s context window you’re using. Claude automatically compacts when needed, or you can run `/compact` manually.

- **Extended thinking**: lets Claude spend more time reasoning through complex problems. Toggle it on via the command menu (`/`). Claude's reasoning appears in the conversation as collapsed blocks: click a block to read it, or press `Ctrl+O` to expand or collapse every thinking block in the session. See **Extended thinking** for details.
- **Multi-line input**: press `Shift+Enter` to add a new line without sending. This also works in the "Other" free-text input of question dialogs.

Reference files and folders

Use @-mentions to give Claude context about specific files or folders. When you type `@` followed by a file or folder name, Claude reads that content and can answer questions about it or make changes to it. Claude Code supports fuzzy matching, so you can type partial names to find what you need:

```
> Explain the logic in @auth (fuzzy matches auth.js,
AuthService.ts, etc.)
> What's in @src/components/ (include a trailing slash for
folders)
```

For large PDFs, you can ask Claude to read specific pages instead of the whole file: a single page, a range like pages 1-10, or an open-ended range like page 3 onward.

When you select text in the editor, Claude can see your highlighted code automatically. The prompt box footer shows how many lines are selected. Press `Option+K` (Mac) / `Alt+K` (Windows/Linux) to insert an @-mention with the file path and line numbers (e.g., `@app.ts#5-10`). Click the selection indicator to toggle whether Claude can see your highlighted text - the eye-slash icon means the selection is hidden from Claude.

You can also hold `Shift` while dragging files into the prompt box to add them as attachments. Click the X on any attachment to remove it from context.

Resume past conversations

Click the **Session history** button at the top of the Claude Code panel to access your conversation history. You can search by keyword or browse by time (Today, Yesterday, Last 7 days, etc.). Click any conversation to resume it with the full message history. New sessions receive AI-generated titles based on your first message. Hover over a session to reveal rename and remove actions: rename to give it a descriptive title, or remove to delete it from the list. For more on resuming sessions, see

Manage sessions.

Resume cloud sessions from Claude.ai

If you use **Claude Code on the web**, you can resume those cloud sessions directly in VS Code. This requires signing in with **Claude.ai Subscription**, not Anthropic Console.

1. **Open session history**

   Click the **Session history** button at the top of the Claude Code panel.

2. **Select the Remote tab**

   The dialog shows two tabs: Local and Remote. Click **Remote** to see sessions from claude.ai.

3. **Select a session to resume**

   Browse or search your cloud sessions. Click any session to download it and continue the conversation locally.

> Only web sessions started with a GitHub repository appear in the Remote tab. Resuming loads the conversation history locally; changes are not synced back to claude.ai.

Check account and usage

Run `/usage` from the command menu to open the Account & usage dialog. It shows your signed-in account, plan, and usage bars for the current session and week with how long until each limit resets.

The dialog also breaks down what is contributing to your plan limits. It flags behaviors that account for 10% or more of recent usage, such as cache misses, long context, and subagent-heavy or highly parallel sessions, each with a tip to reduce it. Attribution tables show how much usage came from each skill, subagent, plugin, and MCP server. Requires Claude Code v2.1.174 or later.

Use the Day and Week toggle to switch between the last 24 hours and the last 7 days. The figures are approximate and computed from local sessions on this machine, so usage from other devices or claude.ai is not included. For more on tracking and reducing usage, see **Track your costs**.

Customize your workflow

Once you're up and running, you can reposition the Claude panel, run multiple sessions, or switch to terminal mode.

Choose where Claude lives

You can drag the Claude panel to reposition it anywhere in VS Code. Grab the panel's tab or title bar and drag it to:

- **Secondary sidebar**: the right side of the window. Keeps Claude visible while you code.
- **Primary sidebar**: the left sidebar with icons for Explorer, Search, etc.
- **Editor area**: opens Claude as a tab alongside your files. Useful for side tasks.

> Use the sidebar for your main Claude session and open additional tabs for side tasks. Claude remembers your preferred location. The Activity Bar sessions list icon is separate from the Claude panel: the sessions list is always visible in the Activity Bar, while the Claude panel icon only appears there when the panel is docked to the left sidebar.

Run multiple conversations

Use **Open in New Tab** or **Open in New Window** from the Command Palette to start additional conversations. Each conversation maintains its own history and context, allowing you to work on different tasks in parallel.

When using tabs, a small colored dot on the spark icon indicates status: blue means a permission request is pending, orange means Claude finished while the tab was hidden.

Switch to terminal mode

By default, the extension opens a graphical chat panel. If you prefer the CLI-style interface, open the **Use Terminal setting** and check the box.

You can also open VS Code settings (`Cmd+,` on Mac or `Ctrl+,` on Windows/Linux), go to Extensions → Claude Code, and check **Use Terminal**.

Manage plugins

The VS Code extension includes a graphical interface for installing and managing **plugins**. Type `/plugins` in the prompt box to open the **Manage plugins** interface.

Install plugins

The plugin dialog shows two tabs: **Plugins** and **Marketplaces**.

In the Plugins tab:

- **Installed plugins** appear at the top with toggle switches to enable or disable them
- **Available plugins** from your configured marketplaces appear below
- Search to filter plugins by name or description
- Click **Install** on any available plugin

When you install a plugin, choose the installation scope:

- **Install for you**: available in all your projects (user scope)
- **Install for this project**: shared with project collaborators (project scope)
- **Install locally**: only for you, only in this repository (local scope)

Manage marketplaces

Switch to the **Marketplaces** tab to add or remove plugin sources:

- Enter a GitHub repo, URL, or local path to add a new marketplace
- Click the refresh icon to update a marketplace's plugin list
- Click the trash icon to remove a marketplace

After making changes, a banner prompts you to restart Claude Code to apply the updates.

> Plugin management in VS Code uses the same CLI commands under the hood. Plugins and marketplaces you configure in the extension are also available in the CLI, and vice versa.

For more about the plugin system, see **Plugins** and **Plugin marketplaces**.

Automate browser tasks with Chrome

Connect Claude to your Chrome browser to test web apps, debug with console logs, and automate browser workflows without leaving VS Code. This requires the **Claude in Chrome extension** version 1.0.36 or higher.

Type `@browser` in the prompt box followed by what you want Claude to do:

```
@browser go to localhost:3000 and check the console for errors
```

You can also open the attachment menu to select specific browser tools like opening a new tab or reading page content.

Claude opens new tabs for browser tasks and shares your browser's login state, so it can access any site you're already signed into.

For setup instructions, the full list of capabilities, and troubleshooting, see **Use Claude Code with Chrome**.

VS Code commands and shortcuts

Open the Command Palette (`Cmd+Shift+P` on Mac or `Ctrl+Shift+P` on Windows/Linux) and type "Claude Code" to see all available VS Code commands for the Claude Code extension.

Some shortcuts depend on which panel is "focused" (receiving keyboard input). When your cursor is in a code file, the editor is focused. When your cursor is in Claude's prompt box, Claude is focused. Use `Cmd+Esc` / `Ctrl+Esc` to toggle between them.

> These are VS Code commands for controlling the extension. Not all built-in Claude Code commands are available in the extension. See **VS Code extension vs. Claude Code CLI** for details.

Command	Shortcut	Description
Focus Input	`Cmd+Esc` (Mac) / `Ctrl+Esc` (Windows/Linux)	Toggle focus between editor and Claude
Open in Side Bar	-	Open Claude in the left sidebar
Open in Terminal	-	Open Claude in terminal mode

Command	Shortcut	Description
Open in New Tab	`Cmd+Shift+Esc` (Mac) / `Ctrl+Shift+Esc` (Windows/Linux)	Open a new conversation as an editor tab
Open in New Window	-	Open a new conversation in a separate window
New Conversation	`Cmd+N` (Mac) / `Ctrl+N` (Windows/Linux)	Start a new conversation. Requires Claude to be focused and `enableNewConversationShortcut` set to `true`
Reopen Closed Session	`Cmd+Shift+T` (Mac) / `Ctrl+Shift+T` (Windows/Linux)	Reopen the most recently closed Claude session tab. Falls through to VS Code's normal reopen-closed-editor when the last closed tab wasn't a Claude session. Disable with `enableReopenClosedSessionShortcut`
Insert @-Mention Reference	`Option+K` (Mac) / `Alt+K` (Windows/Linux)	Insert a reference to the current file and selection (requires editor to be focused)
Show Logs	-	View extension debug logs
Logout	-	Sign out of your Anthropic account

Launch a VS Code tab from other tools

The extension registers a URI handler at `vscode://anthropic.claude-code/open`. Use it to open a new Claude Code tab from your own tooling: a shell alias, a browser bookmarklet, or any script that can open a URL. If VS Code isn't already running, opening the URL launches it first. If VS Code is already running, the URL opens in whichever window is currently focused.

Invoke the handler with your operating system's URL opener.

macOS

```
open "vscode://anthropic.claude-code/open"
```

Linux

```
xdg-open "vscode://anthropic.claude-code/open"
```

Windows

In PowerShell:

```
Start-Process "vscode://anthropic.claude-code/open"
```

In `cmd.exe`, `start` treats its first quoted argument as a window title, so pass an empty title before the URL:

```
start "" "vscode://anthropic.claude-code/open"
```

The handler accepts two optional query parameters:

Parameter	Description
`prompt`	Text to pre-fill in the prompt box. Must be URL-encoded. The prompt is pre-filled but not submitted automatically.
`session`	A session ID to resume instead of starting a new conversation. The session must belong to the workspace currently open in VS Code. If the session isn't found, a fresh conversation starts instead. If the session is already open in a tab, that tab is focused. To capture a session ID programmatically, see **Continue conversations**.

For example, to open a tab pre-filled with "review my changes":

```
vscode://anthropic.claude-code/open?prompt=review%20my%20changes
```

To launch a terminal session instead of a VS Code tab, use the CLI's `claude-cli://` handler. See **Launch sessions from links**.

Configure settings

The extension has two types of settings:

- **Extension settings** in VS Code: control the extension's behavior within VS Code. Open with `Cmd+,` (Mac) or `Ctrl+,` (Windows/Linux), then go to Extensions → Claude Code. You can also type `/` and select **General Config** to open settings.
- **Claude Code settings** in `~/.claude/settings.json` : shared between the extension and CLI. Use for allowed commands, environment variables, hooks, and MCP servers. See **Settings** for details.

Add `"$schema": "https://json.schemastore.org/claude-code-settings.json"` to your `settings.json` to get autocomplete and inline validation for all available settings directly in VS Code.

Extension settings

Setting	Default	Description
`useTerminal`	`false`	Launch Claude in terminal mode instead of graphical panel
`initialPermissionMode`	`default`	Controls approval prompts for new conversations: `default` , `plan` , `acceptEdits` , or `bypassPermissions` . See **permission modes**.
`preferredLocation`	`panel`	Where Claude opens: `sidebar` (right) or `panel` (new tab)
`autosave`	`true`	Auto-save files before Claude reads or writes them
`useCtrlEnterToSend`	`false`	Use Ctrl/Cmd+Enter instead of Enter to send prompts
`enableNewConversationShortcut`	`false`	Enable Cmd/Ctrl+N to start a new conversation
`enableReopenClosedSessionShortcut`	`true`	Use Cmd/Ctrl+Shift+T to reopen the most recently closed Claude session tab. When the last closed tab wasn't a Claude session, the shortcut runs VS Code's normal reopen-closed-editor command instead.
`hideOnboarding`	`false`	Hide the onboarding checklist (graduation cap icon)
`respectGitIgnore`	`true`	Exclude .gitignore patterns from file searches
`usePythonEnvironment`	`true`	Activate the workspace's Python environment when running Claude. Requires the Python extension.
`environmentVariables`	`[]`	Set environment variables for the Claude process. Use Claude Code settings instead for shared config.

Setting	Default	Description
`disableLoginPrompt`	`false`	Skip authentication prompts (for third-party provider setups)
`allowDangerouslySkipPermissions`	`false`	Adds Bypass permissions to the mode selector. Use it only in sandboxes with no internet access.
`claudeProcessWrapper`	-	Executable used to launch the Claude process. The bundled binary path is passed as an argument when present. Set this to a separately installed `claude` binary if the extension build doesn't include one for your platform.

VS Code extension vs. Claude Code CLI

Claude Code is available as both a VS Code extension (graphical panel) and a CLI (command-line interface in the terminal). Some features are only available in the CLI. If you need a CLI-only feature, run `claude` in VS Code's integrated terminal. This requires the **standalone CLI install**: the extension does not add `claude` to your PATH. See **Run CLI in VS Code**.

Feature	CLI	VS Code Extension
Commands and skills	All	Subset (type `/` to see available)
MCP server config	Yes	Partial (add servers via CLI; manage existing servers with `/mcp` in the chat panel)
Checkpoints	Yes	Yes
`!` bash shortcut	Yes	No
Tab completion	Yes	No

Rewind with checkpoints

The VS Code extension supports checkpoints, which track Claude's file edits and let you rewind to a previous state. Hover over any message to reveal the rewind button, then choose from three options:

- **Fork conversation from here**: start a new conversation branch from this message while keeping all code changes intact
- **Rewind code to here**: revert file changes back to this point in the conversation while keeping the full conversation history
- **Fork conversation and rewind code**: start a new conversation branch and revert file changes to this point

For full details on how checkpoints work and their limitations, see **Checkpointing**.

Run CLI in VS Code

To use the CLI while staying in VS Code, open the integrated terminal (`Ctrl+`` ` on Windows/Linux or `Cmd+`` ` on Mac) and run `claude`. The CLI automatically integrates with your IDE for features like diff viewing and diagnostic sharing.

Installing the extension does not put `claude` on your shell PATH. The extension bundles a private copy of the CLI for its chat panel, but typing `claude` in a terminal requires the **standalone CLI install**. Run the install once and the commands on this page, including `claude mcp add` and `claude --resume`, work in any terminal. If `claude` is still not found after installing, **verify your PATH**.

If using an external terminal, run `/ide` inside Claude Code to connect it to VS Code.

Switch between extension and CLI

The extension and CLI share the same conversation history. To continue an extension conversation in the CLI, run `claude --resume` in the terminal. This opens an interactive picker where you can search for and select your conversation.

Include terminal output in prompts

Reference terminal output in your prompts using `@terminal:name` where `name` is the terminal's title. This lets Claude see command output, error messages, or logs without copy-pasting.

Monitor background processes

When Claude runs long-running commands, the extension shows progress in the status bar. However, visibility for background tasks is limited compared to the CLI. For better visibility, have Claude output the command so you can run it in VS Code's integrated terminal.

Connect to external tools with MCP

MCP (Model Context Protocol) servers give Claude access to external tools, databases, and APIs.

To add an MCP server, open the integrated terminal (`Ctrl+`` ` or `Cmd+`` ` ) and run `claude mcp add`. The example below adds GitHub's remote MCP server, which authenticates with a **personal access**

token passed as a header:

```
claude mcp add --transport http github
https://api.githubcopilot.com/mcp/ \
  --header "Authorization: Bearer YOUR_GITHUB_PAT"
```

Once configured, ask Claude to use the tools (e.g., “Review PR #456”).

To manage MCP servers without leaving VS Code, type `/mcp` in the chat panel. The MCP management dialog lets you enable or disable servers, reconnect to a server, and manage OAuth authentication. See the **MCP documentation** for available servers.

Work with git

Claude Code integrates with git to help with version control workflows directly in VS Code. Ask Claude to commit changes, create pull requests, or work across branches.

Create commits and pull requests

Claude can stage changes, write commit messages, and create pull requests based on your work:

```
> commit my changes with a descriptive message
> create a pr for this feature
> summarize the changes I've made to the auth module
```

When creating pull requests, Claude generates descriptions based on the actual code changes and can add context about testing or implementation decisions.

Use git worktrees for parallel tasks

Use the `--worktree` (`-w`) flag to start Claude in an isolated worktree with its own files and branch:

```
claude --worktree feature-auth
```

Each worktree maintains independent file state while sharing git history. This prevents Claude

instances from interfering with each other when working on different tasks. For more details, see **Run parallel sessions with Git worktrees**.

Use third-party providers

By default, Claude Code connects directly to Anthropic's API. If your organization uses Amazon Bedrock, Google Vertex AI, or Microsoft Foundry to access Claude, configure the extension to use your provider instead:

1. **Disable login prompt**

   Open the **Disable Login Prompt setting** and check the box.

   You can also open VS Code settings (`Cmd+,` on Mac or `Ctrl+,` on Windows/Linux), search for "Claude Code login", and check **Disable Login Prompt**.

2. **Configure your provider**

   Follow the setup guide for your provider:

   - **Claude Code on Amazon Bedrock**
   - **Claude Code on Google Vertex AI**
   - **Claude Code on Microsoft Foundry**

   These guides cover configuring your provider in `~/.claude/settings.json`, which ensures your settings are shared between the VS Code extension and the CLI.

Security and privacy

Your code stays private. Claude Code processes your code to provide assistance but does not use it to train models. For details on data handling and how to opt out of logging, see **Data and privacy**.

With auto-edit permissions enabled, Claude Code can modify VS Code configuration files (like `settings.json` or `tasks.json`) that VS Code may execute automatically. To reduce risk when working with untrusted code:

- Enable **VS Code Restricted Mode** for untrusted workspaces

- Use manual approval mode instead of auto-accept for edits
- Review changes carefully before accepting them

The built-in IDE MCP server

When the extension is active, it runs a local MCP server that the CLI connects to automatically. This is how the CLI opens diffs in VS Code's native diff viewer, reads your current selection for `@`-mentions, and — when you're working in a Jupyter notebook — asks VS Code to execute cells.

The server is named `ide` and is hidden from `/mcp` because there's nothing to configure. If your organization uses a `PreToolUse` hook to allowlist MCP tools, though, you'll need to know it exists.

Selection and open-file context. While connected, the CLI includes your current editor selection and the path of the active file as context on each prompt you send. The transcript shows a `Selected N lines from <file>` line when this happens. To exclude a sensitive file such as `.env`, add a `Read` **deny rule** for its path. A matching deny rule prevents both the selected text and the open-file notice for that file from reaching Claude.

Transport and authentication. The server binds to `127.0.0.1` on a random high port and is not reachable from other machines. Each extension activation generates a fresh random auth token that the CLI must present to connect. The token is written to a lock file under `~/.claude/ide/` with `0600` permissions in a `0700` directory, so only the user running VS Code can read it.

Tools exposed to the model. The server hosts a dozen tools, but only two are visible to the model. The rest are internal RPC the CLI uses for its own UI — opening diffs, reading selections, saving files — and are filtered out before the tool list reaches Claude.

Tool name (as seen by hooks)	What it does	Writes?
`mcp__ide__getDiagnostics`	Returns language-server diagnostics — the errors and warnings in VS Code's Problems panel. Optionally scoped to one file.	No
`mcp__ide__executeCode`	Runs Python code in the active Jupyter notebook's kernel. See confirmation flow below.	Yes

Jupyter execution always asks first. `mcp__ide__executeCode` can't run anything silently. On each call, the code is inserted as a new cell at the end of the active notebook, VS Code scrolls it into view, and a native Quick Pick asks you to **Execute** or **Cancel**. Cancelling — or dismissing the picker with `Esc` — returns an error to Claude and nothing runs. The tool also refuses outright when there's no active notebook, when the Jupyter extension (`ms-toolsai.jupyter`) isn't installed, or when the

kernel isn't Python.

> The Quick Pick confirmation is separate from `PreToolUse` hooks. An allowlist entry for `mcp__ide__executeCode` lets Claude *propose* running a cell; the Quick Pick inside VS Code is what lets it *actually* run.

Fix common issues

Extension won't install

- Ensure you have a compatible version of VS Code (1.98.0 or later)
- Check that VS Code has permission to install extensions
- Try installing directly from the **VS Code Marketplace**

Spark icon not visible

The Spark icon appears in the **Editor Toolbar** (top-right of editor) when you have a file open. If you don't see it:

1. **Open a file**: The icon requires a file to be open. Having just a folder open isn't enough.
2. **Check VS Code version**: Requires 1.98.0 or higher (Help → About)
3. **Restart VS Code**: Run "Developer: Reload Window" from the Command Palette
4. **Disable conflicting extensions**: Temporarily disable other AI extensions (Cline, Continue, etc.)
5. **Check workspace trust**: The extension doesn't work in Restricted Mode

Alternatively, click "✱ Claude Code" in the **Status Bar** (bottom-right corner). This works even without a file open. You can also use the **Command Palette** (`Cmd+Shift+P` / `Ctrl+Shift+P`) and type "Claude Code".

Cmd+Esc does nothing on macOS

On macOS Tahoe and later, the system Game Overlay shortcut is bound to `Cmd+Esc` by default and intercepts the keypress before it reaches VS Code. To free the shortcut:

1. Open System Settings
2. Go to Keyboard, then Keyboard Shortcuts, then Game Controllers
3. Clear the Game Overlay checkbox

Alternatively, rebind the extension to a different key: open the VS Code **Keyboard Shortcuts editor** (`Cmd+K Cmd+S`), search for `Claude Code: Focus input` , and assign a new binding.

Claude Code never responds

If Claude Code isn't responding to your prompts:

1. **Check your internet connection**: Ensure you have a stable internet connection
2. **Start a new conversation**: Try starting a fresh conversation to see if the issue persists
3. **Try the CLI**: Run `claude` from the terminal to see if you get more detailed error messages

If problems persist, **file an issue on GitHub** with details about the error.

Uninstall the extension

To uninstall the Claude Code extension:

1. Open the Extensions view (`Cmd+Shift+X` on Mac or `Ctrl+Shift+X` on Windows/Linux)
2. Search for "Claude Code"
3. Click **Uninstall**

To also remove extension data and reset all settings, delete the extension's storage directory for your platform.

On macOS:

```
rm -rf ~/Library/"Application
Support"/Code/User/globalStorage/anthropic.claude-code
```

On Linux:

```
rm -rf ~/.config/Code/User/globalStorage/anthropic.claude-code
```

On Windows, in PowerShell:

```
Remove-Item -Recurse -Force
"$env:APPDATA\Code\User\globalStorage\anthropic.claude-code"
```

For additional help, see the **troubleshooting guide**.

Next steps

Now that you have Claude Code set up in VS Code:

- **Explore common workflows** to get the most out of Claude Code
- **Set up MCP servers** to extend Claude's capabilities with external tools. Add servers using the CLI, then manage them with `/mcp` in the chat panel.
- **Configure Claude Code settings** to customize allowed commands, hooks, and more. These settings are shared between the extension and CLI.